Instructions:
1. Run task_2.py to generate the tensor database if not already generated.
2. Press Run All (or restart kernel and run all cells).
3. You will be prompted to provide an image ID and a k value.

Task 3: Implement a program which, given an image ID and a value “k”, returns and visualizes the most similar k images based on each of the visual model –you will select the appropriate distance/similarity measure for each feature model.
For each match, also list the corresponding distance/similarity score.

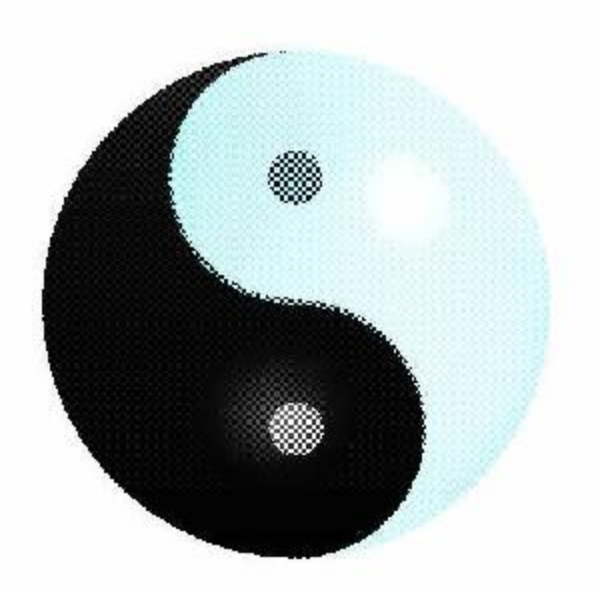
In [ ]:
```
IMG_ID = int(input("Provide an Image ID in the Caltech101 dataset in range [0, 8676]."))

K = int(input("Enter K, the number of similar images K to find for each feature model."))
```

In [ ]:
```
# Initialize dataset, downloads into <git_root>/Code/caltech101 if not present. Untracked by git.

import os
from torchvision.datasets import Caltech101

dataset = Caltech101(root=os.path.abspath(""), download=True)
```

```
Files already downloaded and verified
```

In [ ]:
```
# Find the k most similar images for a given Image ID for each feature model.

image, category = dataset[IMG_ID][0], dataset.categories[dataset[IMG_ID][1]]

if image.mode != 'RGB':
    print("WARNING: The provided image does not have RGB channels, the image will be converted to RGB.")
    image = image.convert('RGB')

print("Input image:\nImage ID: ", IMG_ID)
display(image)
print("Category:", category)
```

```
Input image:
Image ID:  8676
```



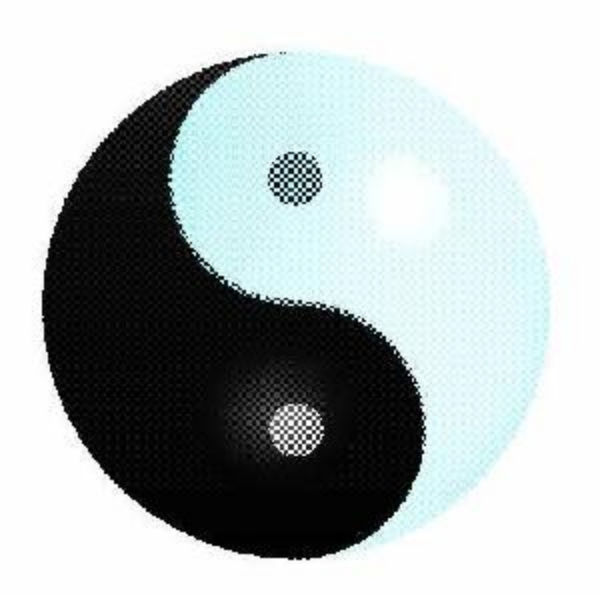
Category: yin_yang

In [ ]:
```
# Resize to 300x100 and partition to grid.

image300x100 = image.resize((300, 100))

from utils.utilities import partition_to_grid

grid = partition_to_grid(image300x100, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

In [ ]:
```
# Color moments (CM10x10).

# Find K most similar images by color moments.

from feature_extractors.color_moments import get_color_vector

color_vector = get_color_vector(grid)

# Load color tensor database into memory.

from utils.utilities import tensor_loader

color_tensor_dict = tensor_loader('color_tensors.pt', os.path.abspath(""))
```

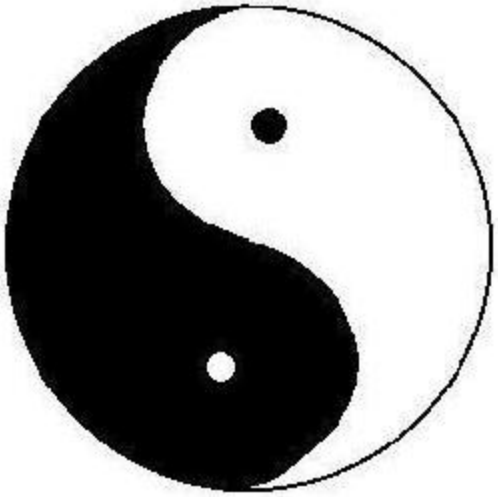
```python
from utils.utilities import top_k_ranker

# Manhattan distance (City-block distance): This distance can be imagined as the length needed
# to move between two points in a grid where you can only move up, down, left or right. This
# measure is sensitive to differences along each dimension of the feature vectors, capturing
# the total absolute difference between corresponding moments, which makes it great for
# assessing differences in color distribution in terms of absolute deviations. Since it
# does not square the differences like Euclidean distance, it is less susceptible outliers
# when many dimensions differ.

# We use Earth Mover's Distance (Wasserstein distance) as color moments can be thought
# of as probability distributions over color space. EMD is capable of comparing distributions
# with different shapes and scales, color moments consider the spatial arrangement of colors,
# not just the frequency of colors. EMD takes this spatial information into account by
# considering how color probability mass is transported from one distribution to another.

from scipy.spatial.distance import cityblock

print("Similar images by Color Moments using distance: Manhattan (cityblock)")

top_k_by_color = top_k_ranker(K, color_vector, color_tensor_dict, cityblock)


for i in range(len(top_k_by_color)):

    id = top_k_by_color[i][1]
    distance = top_k_by_color[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by Color Moments using distance: Manhattan (cityblock)
1.      Image ID:  8676                 Distance:  0.0
```

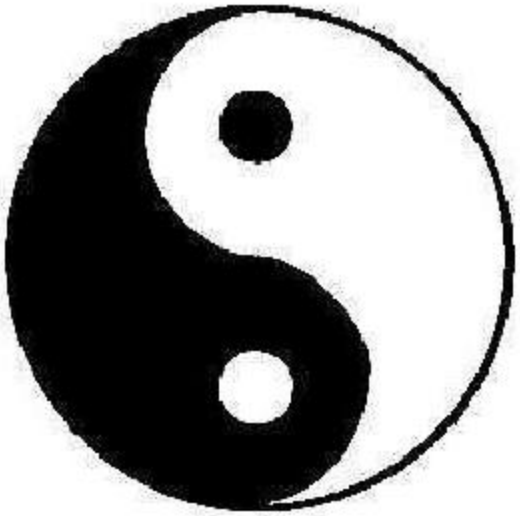


2. Image ID: 8643 Distance: 22167.629

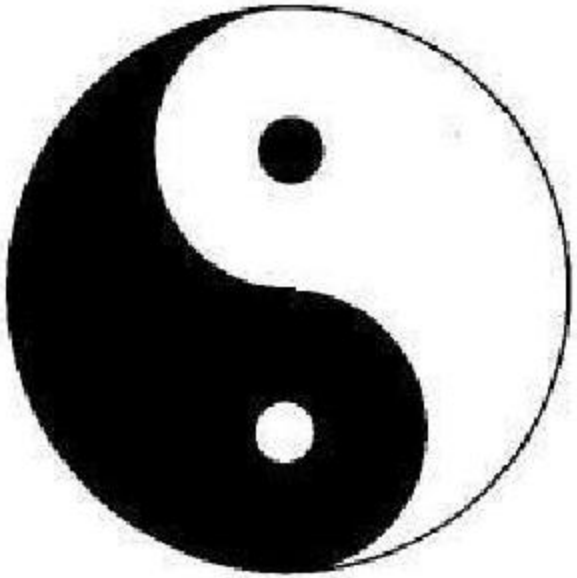
3. Image ID: 8620 Distance: 24031.125



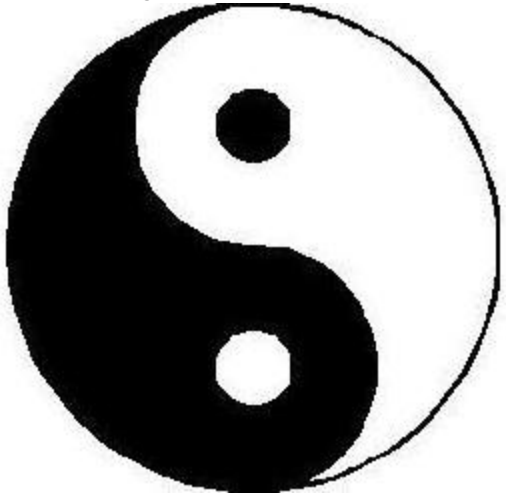
4. Image ID: 8671 Distance: 28903.998



5. Image ID: 8658 Distance: 29158.98



6. Image ID: 8621 Distance: 30705.273



7. Image ID: 8623 Distance: 31181.123

8. Image ID: 5434 Distance: 31207.453

9. Image ID: 8639 Distance: 31974.592

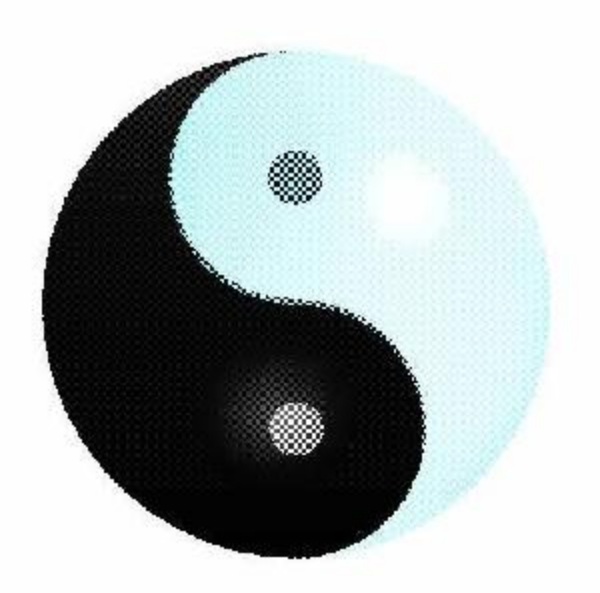
10. Image ID: 6939 Distance: 32399.223

In [ ]:
```
# Convert to grayscale and partition to grid.

from torchvision.transforms import Grayscale
```

```
gs_image = Grayscale()(image300x100)

gs_grid = partition_to_grid(gs_image, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

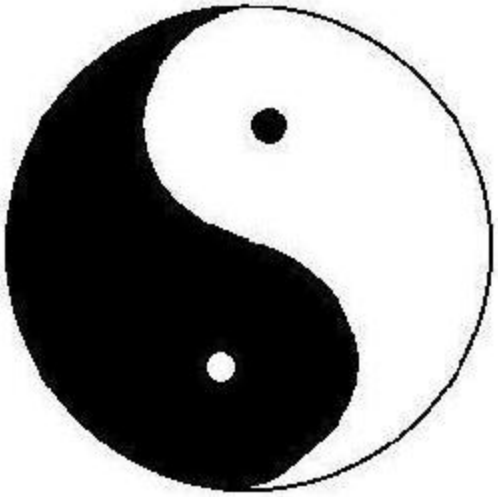
In [ ]:
```
# Histograms of oriented gradients (HOG).

# Find K most similar images by HOG.

from feature_extractors.HOG import get_hog_vector

hog_vector = get_hog_vector(gs_grid)

# Load hog tensor database into memory.

hog_tensor_dict = tensor_loader('hog_tensors.pt', os.path.abspath(""))

# Correlation similarity: This measures the linear relationship between HOG feature vectors,
# and performs well with features that have a strong linear correlation or dependency in
# their gradient orientation distributions, taking into account directionality.

from scipy.spatial.distance import correlation

print("Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation")

top_k_by_hog = top_k_ranker(K, hog_vector, hog_tensor_dict, correlation)


for i in range(len(top_k_by_hog)):

    id = top_k_by_hog[i][1]
    distance = top_k_by_hog[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation
1.      Image ID:  8676                 Distance:  0.0
```

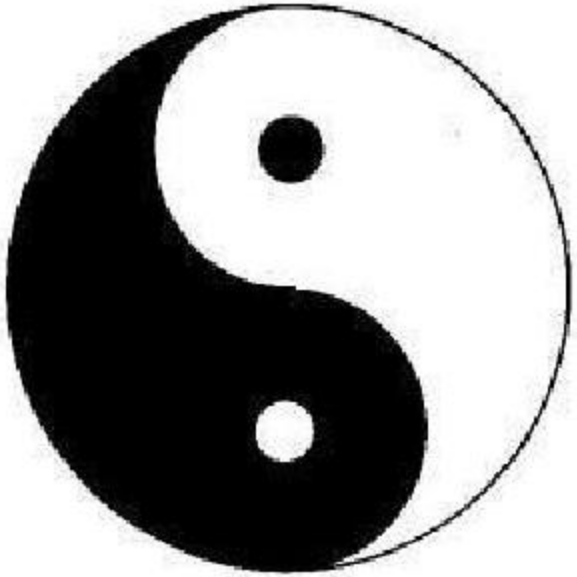


2. Image ID: 8643 Distance: 0.10200639019678914

3. Image ID: 8620 Distance: 0.1027280478371535

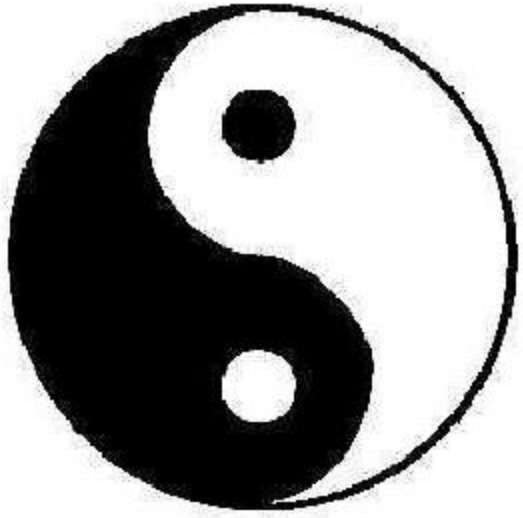


4. Image ID: 8658 Distance: 0.13705439514590978



5. Image ID: 8675 Distance: 0.14870996138508585

6. Image ID: 8671 Distance: 0.15401885443272267

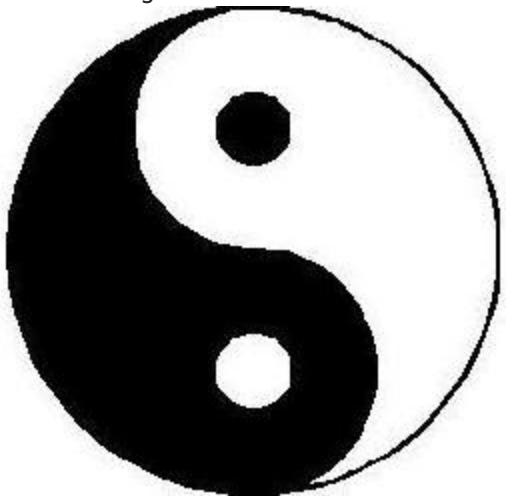


7. Image ID: 8667 Distance: 0.15573586599506828

8. Image ID: 8623 Distance: 0.16617783389556773

9. Image ID: 8644 Distance: 0.178843138963097

10. Image ID: 8621 Distance: 0.17893489881148705

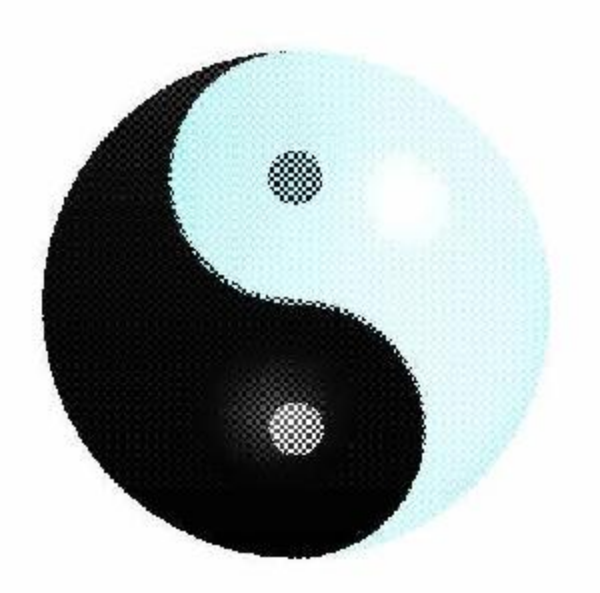


In [ ]:
```
# Resize original image to 224x224 and convert to float tensor.

from torchvision.transforms import transforms

image224x224 = image.resize((224, 224))

img_tensor = transforms.Compose([transforms.ToTensor()])(image224x224)
```

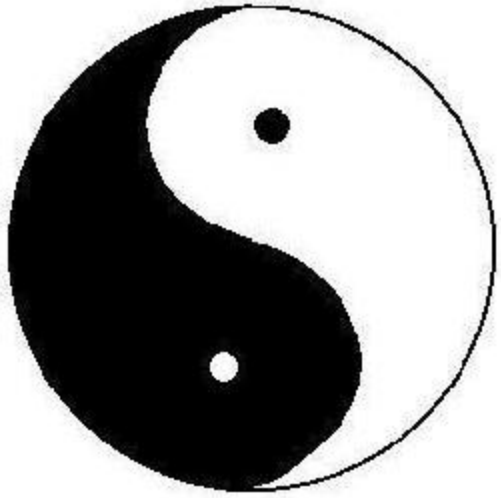
In [ ]:
```
# ResNet pre-processing.

# Find K most similar images by each of the 3 layers chosen from ResNet-50.

from feature_extractors.resnet_50 import get_resnet50_feature_vectors

avgpool_vector, layer3_vector, fc1000_vector = get_resnet50_feature_vectors(img_tensor)

# Load each tensor database into memory

avgpool_tensor_dict = tensor_loader('avgpool_tensors.pt', os.path.abspath(""))
layer3_tensor_dict = tensor_loader('layer3_tensors.pt', os.path.abspath(""))
fc1000_tensor_dict = tensor_loader('fc1000_tensors.pt', os.path.abspath(""))
```

In [ ]:
```
# ResNet-AvgPool-1024.

# Find K most similar images by ResNet-AvgPool-1024.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

from scipy.spatial.distance import cosine

print("Similar images by ResNet-AvgPool-1024 using distance: Cosine")

top_k_by_avgpool = top_k_ranker(K, avgpool_vector, avgpool_tensor_dict, cosine)


for i in range(len(top_k_by_avgpool)):

    id = top_k_by_avgpool[i][1]
    distance = top_k_by_avgpool[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

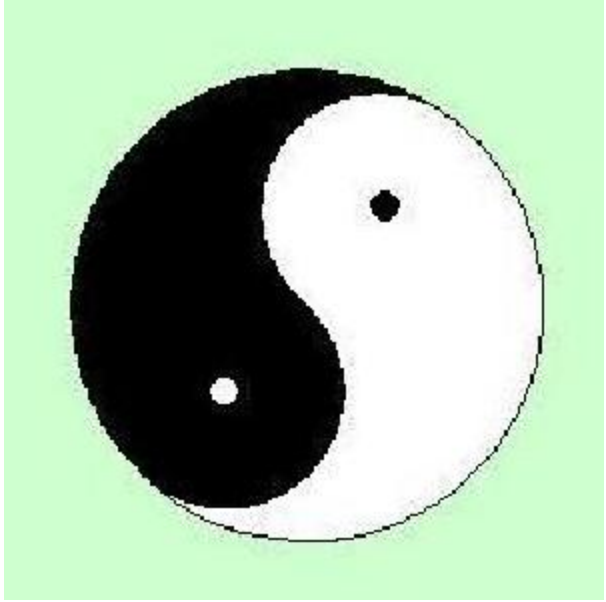
```
Similar images by ResNet-AvgPool-1024 using distance: Cosine
1.      Image ID:  8676                 Distance:  0
```



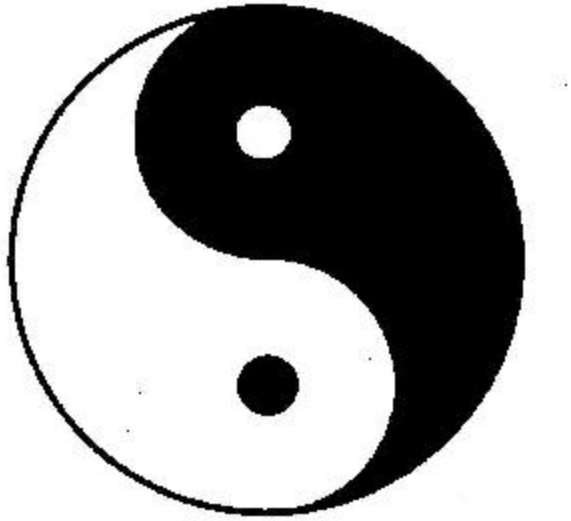
2. Image ID: 8620 Distance: 0.32969796657562256



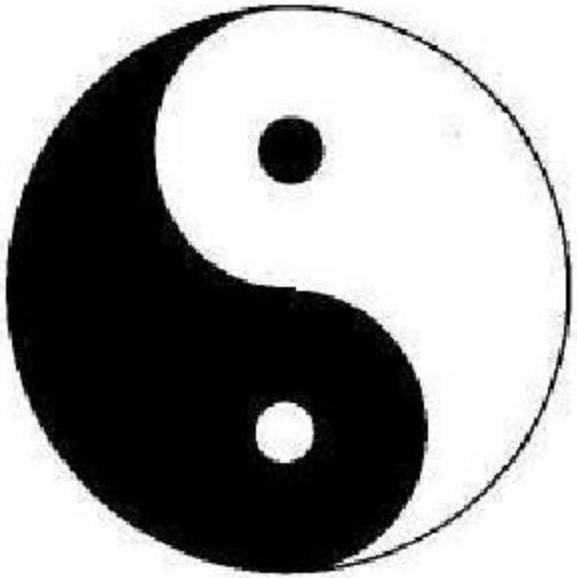
3. Image ID: 8619 Distance: 0.34020692110061646



4. Image ID: 8656 Distance: 0.3500450849533081

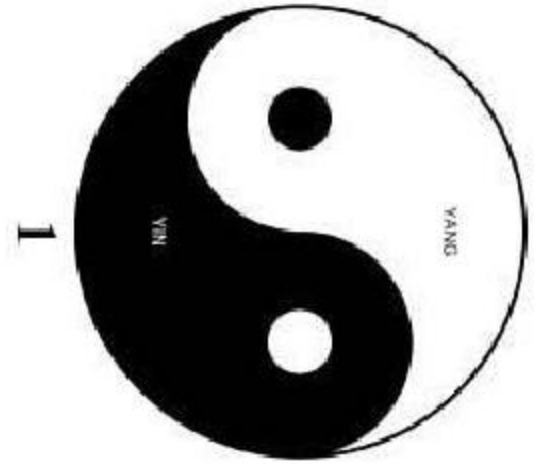


5. Image ID: 8658 Distance: 0.3510517477989197



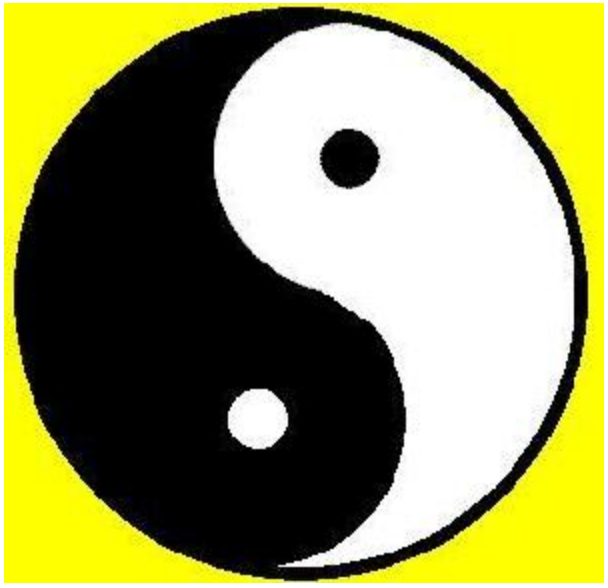
6. Image ID: 8670 Distance: 0.3612073063850403

7. Image ID: 8666 Distance: 0.36177295446395874



8. Image ID: 8632 Distance: 0.3633463382720947

9. Image ID: 8663 Distance: 0.3829125165939331

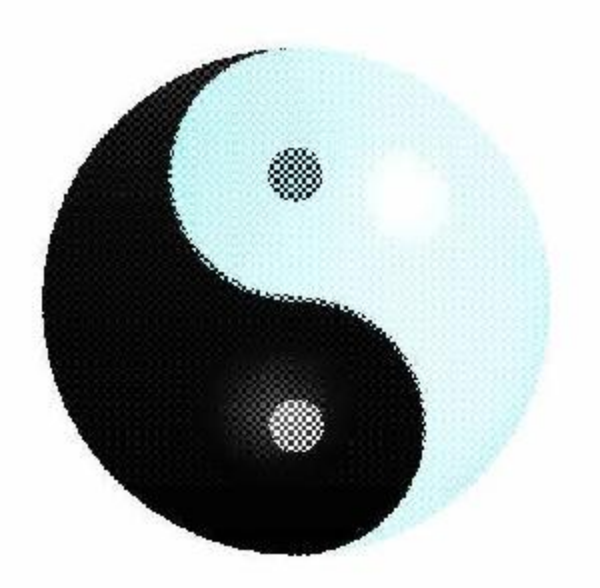


10. Image ID: 8634 Distance: 0.3843638300895691

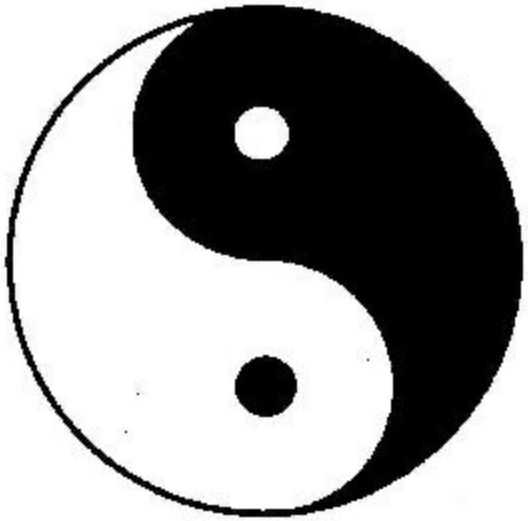
In [ ]:
```
# ResNet-Layer3-1024.

# Find K most similar images by ResNet-Layer3-1024.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
```

```
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

print("Similar images by ResNet-Layer3-1024 using distance: Cosine")

top_k_by_layer3 = top_k_ranker(K, layer3_vector, layer3_tensor_dict, cosine)


for i in range(len(top_k_by_layer3)):

    id = top_k_by_layer3[i][1]
    distance = top_k_by_layer3[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

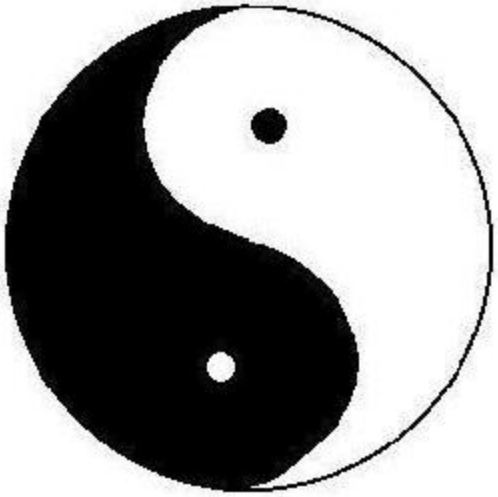
```
Similar images by ResNet-Layer3-1024 using distance: Cosine
1.      Image ID:  8676                 Distance:  0
```



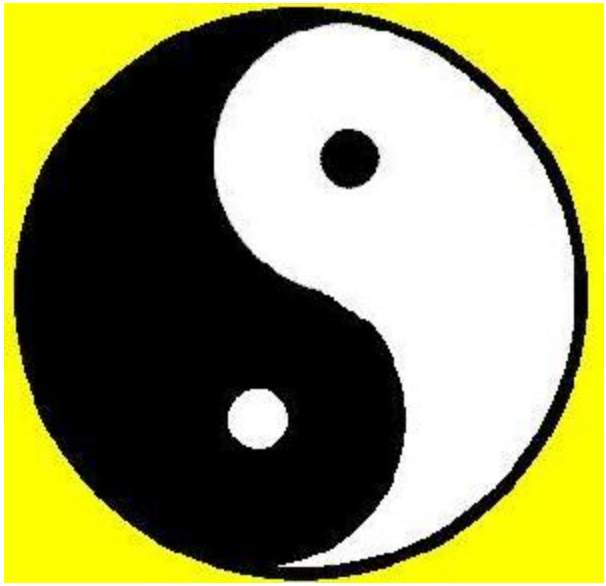
```
2.      Image ID:  8656                 Distance:  0.03968888521194458
```



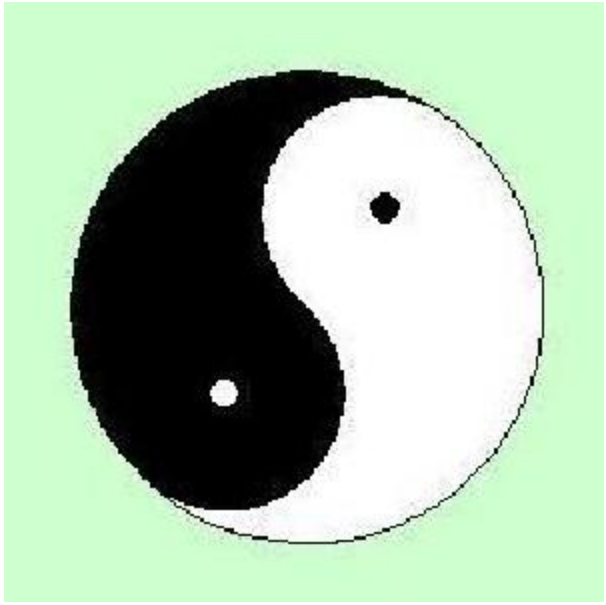
3. Image ID: 8620 Distance: 0.04346984624862671



4. Image ID: 8663 Distance: 0.04572659730911255

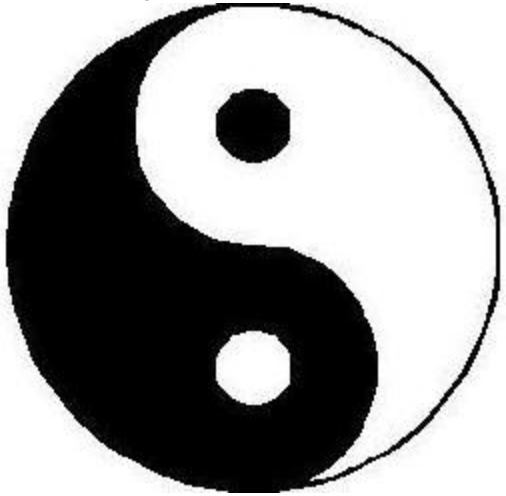


5. Image ID: 8619 Distance: 0.046192049980163574

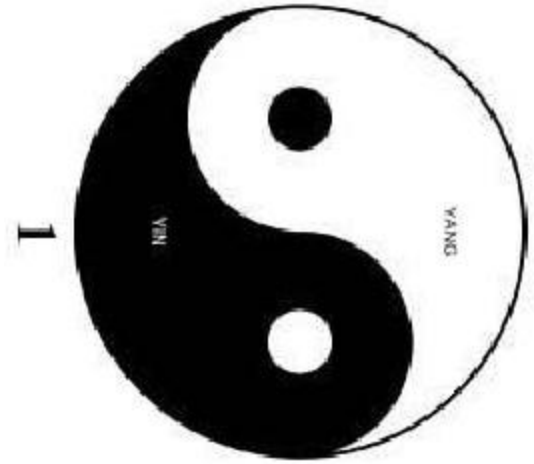


6. Image ID: 8632 Distance: 0.04826962947845459

7. Image ID: 8621 Distance: 0.04918557405471802

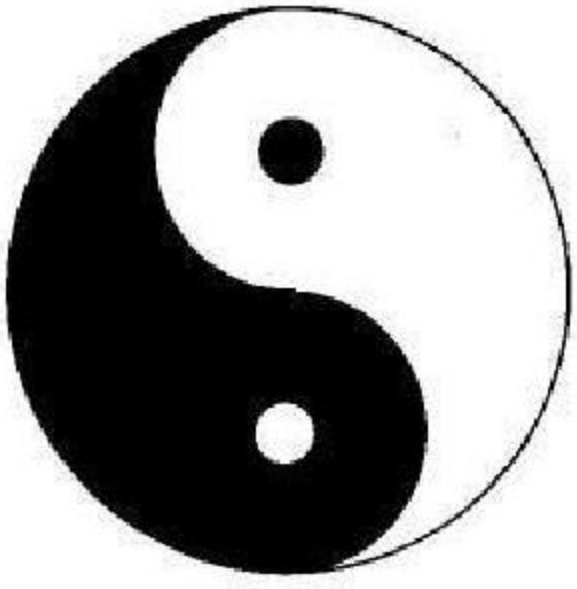


8. Image ID: 8666 Distance: 0.049281179904937744



9. Image ID: 8670 Distance: 0.05097758769989014

10. Image ID: 8658 Distance: 0.05378711223602295



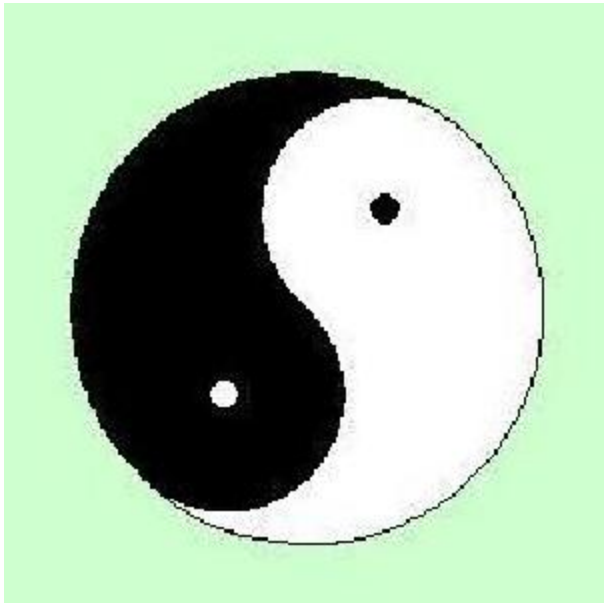
In [ ]:
```
# ResNet-FC-1000.

# Find K most similar images by ResNet-FC-1000.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

print("Similar images by ResNet-FC-1000 using distance: Cosine")

top_k_by_fc1000 = top_k_ranker(K, fc1000_vector, fc1000_tensor_dict, cosine)


for i in range(len(top_k_by_fc1000)):

    id = top_k_by_fc1000[i][1]
    distance = top_k_by_fc1000[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by ResNet-FC-1000 using distance: Cosine

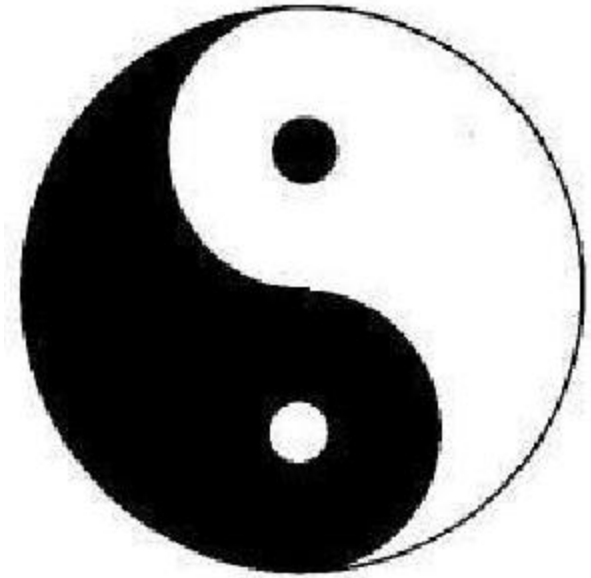
1. Image ID: 8676 Distance: 0

2. Image ID: 8632 Distance: 0.3484852910041809

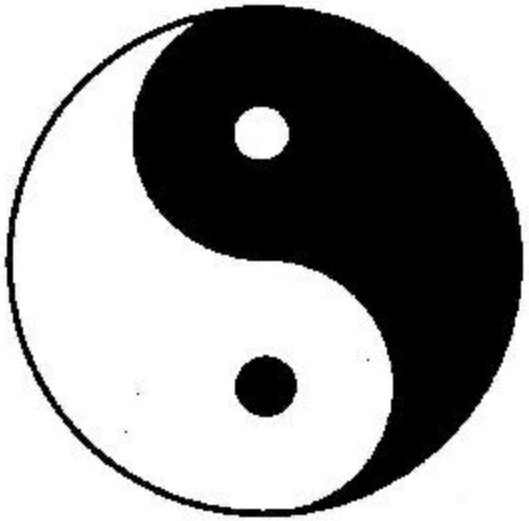
3. Image ID: 8619 Distance: 0.39786356687545776



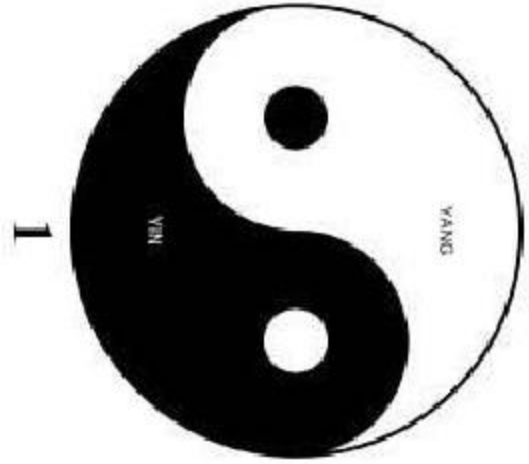
4. Image ID: 8658 Distance: 0.4032387137413025



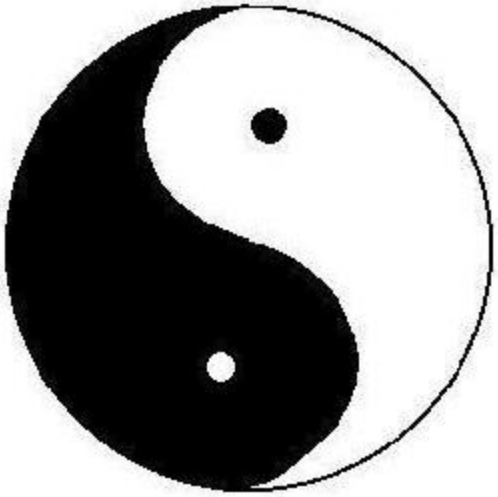
5. Image ID: 8656 Distance: 0.4055728316307068



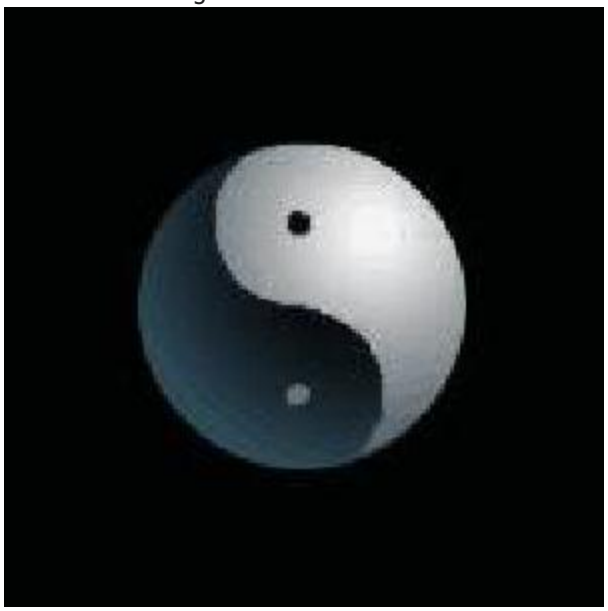
6. Image ID: 8666 Distance: 0.41052669286727905



7. Image ID: 8620 Distance: 0.41611653566360474



8. Image ID: 8668 Distance: 0.4216253161430359



9. Image ID: 8638 Distance: 0.4240154027938843

10. Image ID: 7523 Distance: 0.42618387937545776